



KG COLLEGE OF ARTS AND SCIENCE

Autonomous Institution | Affiliated to Bharathiar University

Accredited with A++ Grade by NAAC

ISO 9001:2015 Certified Institution

KGiSL Campus, Saravanampatti, Coimbatore – 641 035

PROJECT REPORT

Smart Agribot for Tribal Farmers

Submitted to Bharathiar University in partial fulfilment of the requirements for the award of the Degree of
Bachelor of Computer Applications

Submitted by

ANUSRI B

(2322J0317)

Under the Guidance of

Dr.K.S. Maharasan M.C.A, Ph.D.

Assistant Professor

Department of Computer Applications

MARCH 2026

DECLARATION

DECLARATION

I declare that the work which is being presented in this project report entitled “**Smart Agribot for Tribal Farmers**” submitted to the Department of Computer Applications, KG College of Arts and Science, Saravanampatti, Coimbatore for the award of the degree of Bachelor of Computer Applications of the Bharathiar University, is an authentic record of my work carried out under the supervision and guidance of **Dr.K.S. Maharasan M.C.A, Ph.D.**

March 2026

Coimbatore

ANUSRI B

CERTIFICATE

CERTIFICATE

This is to certify that the project entitled “**Smart Agribot for Tribal Farmers**” submitted to Bharathiar University in partial fulfilment for the award of the degree of Bachelor of Computer Applications is a record of original work done by **ANUSRI B (2322J0317)** during the period of study in **KG COLLEGE OF ARTS AND SCIENCE** under the supervision of **Dr.K.S. Maharasan M.C.A, Ph.D., ASSISTANT PROFESSOR, DEPARTMENT OF COMPUTER APPLICATIONS.**

Place: Coimbatore

Date:

SIGNATURE OF THE GUIDE

SIGNATURE OF THE HOD

(College Seal)

Submitted for the Viva-Voce Examination held on _____

INTERNAL EXAMINER

EXTERNAL EXAMINER



Netcon Technologies India Pvt Ltd

New No. 4, Kalapatti Main Road, Civil Aerodrome Post,
Nehru Nagar West, Coimbatore

Website: www.netconglobal.com

Phone: 097513 44000

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Ms. Anusri B (2322J0317)** the student of **KG College of Arts and Science, Coimbatore** studying final year **Bachelor of Computer Applications (BCA)** has been accepted to do project entitled "**Smart Agribot for Tribal Farmers**" in our concern during December 2025 to February 2026.

For Netcon Technologies India Pvt Ltd,



HR Signature

P. Nair

ACKNOWLEDGEMENT

Acknowledgement

I express my sincere thanks to **Dr. Ashok Bakthavathsalam, Managing Trustee** for allowing me to do this course of study and to undertake this project work.

It is my pleasure to express my sincere thanks to **Dr.B. Vanitha, Secretary**, KG College of Arts and Science for her valuable thoughts.

I take this opportunity to convey my sincere thanks to **Dr.S. Vidhya, Principal**, KG College of Arts and Science for her ethical encouragement throughout the course.

I would like to express my gratefulness to **Dr.P. Ajitha, Dean-Curriculum Development Cell**, KG College of Arts and Science, for her untiring support throughout the course.

I take this opportunity to convey my sincere thanks to **Dr.M. Usha, Dean-School of Computational Science**, KG College of Arts and Science for her moral support throughout the course.

I have great pleasure in acknowledging my thanks to **Dr.P. Vijayakumar, Associate Professor & Head, Department of Computer Applications**, KG College of Arts and Science for his encouragement and help throughout the course.

I express my sincere thanks to **Dr.K.S. Maharasan M.C.A, Ph. D, Assistant Professor, Department of Computer Applications** for his constant encouragement and motivation throughout the project. I thank for his endless support and encouragement towards the work.

CONTENTS

S. No	CONTENTS	PAGE NO
	ACKNOWLEDGEMENT	
	CONTENTS	
	SYNOPSIS	
1	INTRODUCTION	1
	1.1 Organization Profile	2
	1.2 System Specification	3
	1.2.1 Hardware Specification	3
	1.2.2 Software Specification	3
2	SYSTEM STUDY	
	2.1 Existing System	4
	2.1.1 Drawbacks of Existing System	4
	2.2 Proposed System	5
	2.2.1 Features of Proposed System	5
3	SYSTEM DESIGN AND DEVELOPMENT	
	3.1 File Design	6
	3.2 Input Design	6
	3.3 Output Design	7
	3.4 Database Design	7
	3.5 System Development	8
	3.6 Description of Modules	8
4	TESTING AND IMPLEMENTATION	9
5	CONCLUSION	11
6	BIBLIOGRAPHY	12
7	APPENDICES	
	A. Data Flow Diagram	13
	B. Table Structure	16
	C. Sample Coding	18
	D. Sample Input	26
	E. Sample Output	29

SYNOPSIS

SYNOPSIS

Agriculture plays a vital role in the livelihood of tribal and small-scale farmers, yet many of them face continuous challenges such as pest attacks, improper crop management, lack of timely irrigation guidance, limited access to government schemes, and insufficient agricultural awareness. Most tribal farmers depend on traditional practices and informal advice, which often leads to crop losses, reduced productivity, and unstable income. The absence of easily accessible, language-friendly, and technology-driven support systems further widens the gap between modern agricultural knowledge and rural farming communities. Addressing these issues requires an integrated and user-friendly digital solution that can support farmers throughout the entire crop lifecycle.

The **Smart Agribot for Tribal Farmers** is an ICT-based multilingual web application developed to assist farmers in making informed agricultural decisions. The system provides guidance on pest detection, crop management practices, irrigation scheduling, and harvesting techniques in a simple and understandable manner. Farmers can access crop-specific recommendations based on their farming stage, symptoms observed, and seasonal conditions. The application also includes information on various government agricultural schemes, helping farmers understand eligibility criteria, benefits, and application procedures without relying on intermediaries. In addition, the system integrates agricultural news and updates through RSS feeds, ensuring farmers stay informed about current developments, weather alerts, and policy changes.

The proposed system is implemented using Python Flask as the backend framework and SQLite as the database, making it lightweight, cost-effective, and suitable for rural deployment. Multilingual support enables farmers to interact with the system in their preferred language, improving usability and adoption among tribal communities. By providing timely, reliable, and localized agricultural assistance, the Smart Agribot aims to reduce crop losses, enhance productivity, increase farmers' income, and contribute to sustainable farming practices. Ultimately, the project seeks to strengthen food security and empower tribal farmers by bridging the gap between modern agricultural technology and traditional farming methods.

1. INTRODUCTION

1.1 ORGANIZATION PROFILE

Today's Business environment is closely aligned with information technology and aid organizations to optimize, business value, enhance business productivity, improve employee productivity and customer satisfaction. They at Netcon understand customer business needs and offer end to end technology solutions, ranging from IT Infrastructure management to Unified communications to enhance our customer's business growth. Their Consulting, Solutions and Services offerings are Innovative, cost effective and scalable to meet customer's requirement. They adopt industry best practices in terms of people, process and technology and ensure maximum customer satisfaction

CORPORATE OVERVIEW

Netcon Technologies India Private Ltd. is promoted and managed by technocrats with a vision for empowering customers with the power of convergence to enhance and optimize business performance. The promoters have vast experience in consulting, deployment and management of Large, complex multi service networks for more than 15 years. They at Netcon are focused on offering leading edge solutions and services ranging from IT infrastructure management to Unified communications. Many of their customers have benefited from solutions deployed by our professionally skilled, certified and dedicated team. They at netcon adopt a professional and personalized approach towards providing quality technology solutions ranging from IT infrastructure management to Convergent Solutions.

SOLUTIONS

Netcon has a team of dedicated, experienced and certified professionals to provide technology consulting in the areas of Networking, Security, Data Center solutions, hosting services and Unified Communication.

- Networking Solutions
- Security Solutions
- Unified Communications
- Data Center Solutions & Mobility Solutions

OVERVIEW OF THE PROJECT

Agriculture remains the backbone of rural livelihoods in India, particularly for tribal and small-scale farmers who depend heavily on farming for their income and food security. Despite their significant contribution to agricultural production, these farmers often face multiple challenges such as pest infestations, unpredictable weather conditions, improper irrigation practices, limited access to modern crop management techniques, and a lack of awareness about government support schemes. In many tribal regions, farmers still rely on traditional knowledge and informal advice, which may not always be sufficient to address current agricultural problems. The digital divide and language barriers further restrict the adoption of modern agricultural technologies among tribal communities.

With the rapid advancement of Information and Communication Technology (ICT), digital platforms have emerged as powerful tools for improving agricultural productivity and decision-making. ICT-based solutions enable farmers to access real-time information, expert recommendations, and best farming practices at low cost. However, most existing agricultural applications are complex, language-restricted, or not specifically designed for the needs of tribal farmers. There is a growing need for a simple, multilingual, and farmer-centric system that can provide end-to-end agricultural assistance, from crop planning to harvesting, in an easily understandable format.

The **Smart Agribot for Tribal Farmers** is proposed as a web-based agricultural assistance system that aims to bridge this gap between technology and grassroots farming communities. The system is designed to support farmers by offering guidance on pest detection, crop management, irrigation scheduling, and harvesting practices. By allowing farmers to interact with the application in their preferred language, the system ensures better accessibility and usability. The inclusion of government scheme information helps farmers become aware of financial aid, subsidies, and welfare programs, reducing dependency on middlemen. Additionally, the integration of agricultural news through RSS feeds keeps farmers informed about recent developments, weather alerts, and policy updates.

Developed using Python Flask and SQLite, the proposed system is lightweight, scalable, and suitable for deployment in rural and semi-urban areas with limited infrastructure. The Smart Agribot emphasizes simplicity, reliability, and practical usability, ensuring that farmers can easily adopt the technology without requiring advanced technical skills. By providing timely, accurate, and localized agricultural recommendations, the system aims to improve crop productivity, reduce losses, enhance farmers' income, and promote sustainable agricultural practices. Ultimately, this project contributes to empowering tribal farmers by enabling informed decision-making and strengthening food security through digital innovation.

OBJECTIVES OF THE STUDY

The primary objective of the Smart Agribot for Tribal Farmers project is to design and develop a simple, multilingual, and user-friendly web application that provides comprehensive agricultural assistance to tribal and small-scale farmers. The system aims to support farmers in making informed decisions by offering timely guidance on various agricultural activities throughout the crop lifecycle. By integrating modern ICT tools with traditional farming practices, the project seeks to improve accessibility to agricultural knowledge and promote sustainable farming methods among rural communities.

Another important objective of the study is to assist farmers in identifying pest-related issues and receiving appropriate recommendations for pest control and crop protection. The system focuses on reducing crop losses by providing early pest detection guidance and suitable preventive measures. In addition, the project aims to provide effective crop management and irrigation planning support based on crop type, growth stage, and seasonal conditions, thereby enhancing productivity and efficient resource utilization.

The project also aims to create awareness among farmers about government agricultural schemes, subsidies, and welfare programs by presenting relevant information in an easily understandable format. This objective helps farmers gain direct access to official support systems and reduces dependency on intermediaries. Furthermore, the inclusion of agricultural news and updates through RSS feeds ensures that farmers remain informed about recent developments, weather alerts, and policy changes affecting the agricultural sector.

The study aims to develop a lightweight and cost-effective system using Python Flask and SQLite, making it suitable for deployment in rural and tribal regions with limited technical

infrastructure. By improving usability through multilingual support and simple interfaces, the project ultimately seeks to increase agricultural productivity, enhance farmers’ income, strengthen food security, and empower tribal farmers through digital innovation.

1.2 SYSTEM SPECIFICATION

1.2.1 Development Requirements

These specifications outline the minimum hardware and software required by the developers to build, test, and maintain the Smart Agribot application.

Table 1: Development Requirements

Category	Specification
Hardware	
Processor	Intel Core 2 Duo (3 GHz) or higher
RAM	4GB RAM
Storage	450 GB SSD
Display	17-inch Monitor or standard laptop display
Peripherals	Standard Keyboard (110 Keys) and Mouse
Software	
Operating System	Windows 8 / 10 / 11, Linux, or macOS
Programming Language	Python 3.x
Server-Side Framework	Flask
Database (Back End)	SQLite3
Front End Technologies	HTML, CSS, JavaScript
Required Libraries	feedparser, werkzeug, sqlite3
Development Tools	Visual Studio Code, PyCharm, or equivalent IDE

1.2.2 Deployment Requirements

These specifications cover the hardware and software required to host and serve the Smart Agribot application, whether deployed locally for a rural community or on a cloud platform (like PythonAnywhere) for broader access.

Table 2: Deployment Requirements

Category	Specification
Hardware	
Server / Hosting Device	Standard Cloud Virtual Machine or Basic Local Server / PC
RAM	1 GB minimum (2 GB recommended)
Storage	Minimum 5 GB free space for application files and database growth
Network	Active Internet Connection (required to fetch dynamic RSS news feeds)
Software	
Operating System	Linux (Ubuntu/Debian) or Windows Server / OS
Application Environment	Python 3.x Runtime
Web Server	Built-in Flask Server (for local) or WSGI Server (e.g., Gunicorn for cloud)
Database Engine	SQLite3 (Built into Python)

1.2.3 End-User Requirements

These specifications define the minimal device requirements for the tribal and small-scale farmers to successfully access and interact with the Smart Agribot application.

Table 3: End-User Requirements

Category	Specification
Hardware	
Device Type	Smartphone, Tablet, Laptop, or Desktop Computer
Processor	Basic Mobile or Desktop Processor
RAM	Minimum 2 GB RAM
Software	
Operating System	Android, iOS, Windows, macOS, or Linux
User Interface	Modern Web Browser (Google Chrome, Mozilla Firefox, Safari, Edge, etc.)
Connectivity	Internet data connection (Mobile Data/Wi-Fi) to access the web portal and updates

FRONT END FEATURES

The front end of the Smart Agribot for Tribal Farmers is designed to provide a simple, clean, and user-friendly interface that can be easily understood by tribal and small-scale farmers with limited digital literacy. The application interface is developed using HTML and CSS, ensuring lightweight performance and compatibility with basic systems commonly available in rural areas. The layout follows a structured and intuitive design pattern where major modules such as Pest Detection, Crop Management, Irrigation Guidance, Harvesting Advice, Government Schemes, and Agricultural News are clearly accessible through a well-organized navigation menu. This reduces confusion and allows farmers to quickly access the required module without unnecessary complexity.

One of the major front-end strengths of the system is multilingual support. The interface dynamically changes content based on the selected language preference of the farmer, typically English or Tamil. All labels, headings, instructions, and recommendations are displayed in the chosen language, which significantly improves accessibility and adoption among tribal communities. The translation logic is integrated into the template structure, ensuring consistency across all modules. This feature eliminates language barriers and enhances the usability of digital agricultural assistance systems.

The user registration and login interface includes proper form validations to ensure data accuracy. Input validation is implemented for fields such as mobile number, email address, and password strength. Required fields cannot be left blank, and duplicate entries are prevented through validation mechanisms. This improves data integrity and ensures secure access to personalized dashboards. Dropdown menus are used for crop selection, season selection, soil type, and growth stages to minimize typing errors and standardize user inputs. The use of combobox elements enhances user interaction and simplifies data entry.

The dashboard design provides a personalized experience for each farmer. After login, users can view recent activities, previously accessed recommendations, and updated agricultural news. The layout ensures that important information is visually separated using panels and card-based design structures. Result displays for irrigation schedules, pest solutions, and crop management recommendations are formatted clearly, making them easy to read and understand.

Only relevant dropdown options are displayed based on user selection, preventing confusion caused by irrelevant choices.

Interactive elements such as dynamic dropdown filtering improve the overall user experience. For example, in the irrigation module, the season and soil type dropdowns display only the options relevant to the selected crop. This prevents unnecessary “no result” messages and ensures smooth navigation. The system avoids displaying technical errors to users and instead provides clean, readable outputs. Responsive design principles are followed so that the application can function properly on desktops and mobile devices, which is important for rural accessibility.

The front end emphasizes simplicity, clarity, multilingual accessibility, and structured navigation. The visual design supports farmers in making informed agricultural decisions without overwhelming them with technical complexity. The interface ensures smooth interaction between the user and the backend processing system, forming a strong foundation for the Smart Agribot application.

BACK END FEATURES

The backend of the Smart Agribot system is developed using Python Flask as the server-side framework and SQLite as the database management system. Flask provides a lightweight and flexible web framework suitable for rural deployment environments where system resources may be limited. The backend architecture follows a modular structure where each agricultural functionality such as pest detection, crop management, irrigation scheduling, harvesting guidance, and scheme information is implemented through separate routes and processing logic.

The database design ensures structured storage of all agricultural data. Tables such as users, pest_guidance, crop_management, irrigation_rules, harvest_guidance, schemes, action_history, and rss_sources store the relevant information in an organized manner. Proper relational mapping using primary keys and foreign keys ensures data consistency and integrity. User authentication is securely managed using password hashing techniques rather than storing plain-text passwords. Session management is implemented to maintain login states securely and restrict unauthorized access. One of the key backend features is the rule-based recommendation engine. Based on user inputs such as crop type, symptom selection, season, and soil type, the

backend queries the database and retrieves appropriate guidance. This decision logic ensures that farmers receive precise and context-specific recommendations rather than generic advice.

In the irrigation module, the backend filters schedules based on crop, season, and soil combinations, ensuring that only valid data combinations produce results. If no matching record exists, the system simply returns no output instead of displaying error messages. The system integrates dynamic RSS feed fetching using the feedparser library. The `rss_sources` table stores RSS URLs, and during dashboard loading, the backend fetches live agricultural news from external websites. This ensures that farmers receive updated information without manual database updates. The backend dynamically parses XML feed data and extracts titles, links, and publication dates before rendering them on the dashboard.

The multilingual support mechanism is implemented at the backend level using translation dictionaries and language session variables. Based on the selected language stored in the user session, the backend dynamically retrieves either English or Tamil content from database fields such as `schedule_en` and `schedule_ta`. This ensures seamless switching between languages without duplicating templates.

The backend also includes history tracking functionality. Each user action such as pest lookup, irrigation query, or crop recommendation is recorded in the `action_history` table along with timestamps and selected language. This feature allows tracking of usage patterns and provides a personalized experience in the dashboard. The architecture is scalable and can support future enhancements such as image-based pest detection, machine learning integration, or admin management modules.

The backend of the Smart Agribot system ensures secure authentication, structured data management, dynamic recommendation processing, multilingual adaptability, and real-time RSS integration. It forms the core processing engine that transforms user inputs into meaningful agricultural guidance, ensuring reliability, efficiency, and scalability of the system.

SYSTEM STUDY

2. SYSTEM STUDY

2.1 EXISTING SYSTEM

In the existing agricultural support system, tribal and small-scale farmers largely depend on traditional farming practices, personal experience, and informal guidance from fellow farmers or local input dealers. Agricultural decisions related to pest control, crop management, irrigation, and harvesting are often based on trial-and-error methods rather than scientific analysis. While government agricultural departments and extension services provide support through field officers and awareness programs, such assistance is limited in reach and frequency, especially in remote tribal regions. As a result, many farmers do not receive timely or accurate information required to address emerging agricultural challenges.

Some farmers rely on manual advisory methods such as pamphlets, radio broadcasts, television programs, or periodic agricultural meetings to gain knowledge. Although these sources provide useful information, they are not personalized to individual farmers' crops, soil conditions, or farming stages. Moreover, language barriers and lack of digital literacy restrict tribal farmers from fully benefiting from existing online agricultural portals and mobile applications. Many available digital platforms are complex, available only in limited languages, and require continuous internet access, making them unsuitable for farmers in rural and tribal areas.

In the current system, pest detection and crop disease identification are often carried out through visual inspection or consultation with local vendors, which may lead to incorrect diagnosis and excessive use of pesticides. Irrigation practices are generally based on fixed schedules or traditional beliefs rather than crop-specific and seasonal requirements, resulting in water wastage or crop stress. Information about government schemes and subsidies is often poorly communicated, causing eligible farmers to miss valuable financial support. Overall, the existing system lacks an integrated, user-friendly, and farmer-centric digital solution that can provide end-to-end agricultural guidance in a timely and accessible manner.

2.1.1 Drawbacks of the Existing System

- The existing agricultural support system faces several serious limitations that directly affect tribal and small-scale farmers. One major drawback is the heavy dependence on traditional knowledge and informal advice from fellow farmers or local vendors, which may not always be scientifically accurate or suitable for changing climatic conditions. This often leads to delayed identification of crop diseases and pests, resulting in avoidable crop losses and reduced productivity.
- Another significant drawback is the lack of timely and personalized guidance. Farmers usually receive generalized advice through pamphlets, radio programs, or occasional agricultural meetings, but these sources do not consider specific crop types, soil conditions, or growth stages. As a result, farmers are unable to make precise decisions tailored to their individual farm requirements.
- The absence of a centralized digital platform further limits access to reliable agricultural information. Tribal farmers in remote areas often struggle to obtain updated recommendations related to irrigation scheduling, fertilizer usage, and modern crop management practices. This information gap widens the divide between advanced agricultural research and grassroots farming communities.
- Irrigation practices in the current system are largely based on traditional beliefs rather than scientific recommendations. Farmers may over-irrigate or under-irrigate crops due to lack of proper scheduling guidance, leading to water wastage, soil degradation, and lower crop yield. This inefficient water management negatively impacts both productivity and sustainability.
- Another drawback is the limited awareness about government agricultural schemes and subsidies. Many eligible farmers are unaware of financial assistance programs due to poor communication and lack of structured information dissemination. This results in underutilization of welfare schemes intended to support rural communities.
- The existing system also lacks real-time updates regarding weather alerts, policy changes, and market trends. Farmers often make decisions without current information, increasing their exposure to risks caused by sudden climatic variations or price fluctuations.

2.2 PROPOSED SYSTEM

The proposed system, **Smart Agribot for Tribal Farmers**, is an ICT-based multilingual web application designed to provide comprehensive agricultural assistance to tribal and small-scale farmers. The system aims to overcome the limitations of the existing agricultural practices by offering an integrated digital platform that supports farmers throughout the entire crop lifecycle. By combining modern information technology with agricultural knowledge, the proposed system delivers timely, accurate, and localized recommendations in a simple and user-friendly manner.

The Smart Agribot system provides key functionalities such as pest detection guidance, crop management support, irrigation scheduling, and harvesting assistance. Farmers can interact with the system by selecting crop types, growth stages, and observed symptoms, based on which the system generates appropriate recommendations and preventive measures. The inclusion of multilingual support ensures that farmers can access information in their preferred language, reducing communication barriers and improving usability among tribal communities.

In addition to farming guidance, the proposed system includes a dedicated module for government agricultural schemes, enabling farmers to easily access information related to subsidies, eligibility criteria, benefits, and application procedures. This feature helps farmers stay informed about available welfare programs and reduces dependency on intermediaries. The system also integrates agricultural news and updates through RSS feeds, providing real-time information on weather alerts, policy changes, and market-related developments.

The proposed system is developed using Python Flask for backend processing and SQLite for data storage, making it lightweight, cost-effective, and suitable for deployment in rural environments with limited infrastructure. By offering an easy-to-navigate interface, reliable recommendations, and centralized agricultural information, the Smart Agribot aims to reduce crop losses, improve productivity, enhance farmers' income, and promote sustainable agricultural practices. Ultimately, the proposed system seeks to empower tribal farmers by bridging the gap between traditional farming methods and modern digital technologies.

2.2.1 Advantages of proposed system

- The proposed Smart Agribot system provides a centralized digital platform that integrates pest detection, crop management, irrigation guidance, harvesting advice, government schemes, and agricultural news in one place.
- It offers multilingual support, enabling tribal farmers to access information in their preferred language, thereby improving understanding and usability.
- The system delivers crop-specific and stage-wise recommendations, helping farmers make accurate and informed decisions throughout the crop lifecycle.
- Timely pest identification and preventive guidance reduce crop damage and minimize excessive pesticide usage.
- Efficient irrigation scheduling promotes optimal water utilization, reduces wastage, and supports sustainable farming practices.
- The inclusion of government scheme information increases awareness about subsidies and financial assistance, reducing dependency on intermediaries.
- Real-time agricultural news and weather updates through RSS feeds keep farmers informed about current developments and risks.
- The lightweight design using Python Flask and SQLite ensures cost-effectiveness and suitability for rural deployment with limited infrastructure.
- The personalized dashboard and history tracking feature allow farmers to review previous recommendations and monitor their farming activities effectively.
- The system supports both Tamil and English languages, making it accessible to tribal and rural farmers who may not be comfortable with English-only applications.

Features of the Proposed System

The Smart Agribot for Tribal Farmers is designed with several key features that collectively provide comprehensive agricultural support in a simple and accessible manner. One of the primary features of the system is multilingual support, which allows farmers to interact with the application in their preferred regional language. This feature significantly improves usability among tribal farmers by eliminating language barriers and enabling better understanding of agricultural guidance.

Another important feature is pest detection assistance, where farmers can identify pest-related issues by selecting crop types and observed symptoms. Based on the inputs provided, the system offers suitable recommendations, preventive measures, and control methods. This helps farmers take timely action, reducing crop damage and minimizing excessive use of pesticides. The system also maintains a history of pest reports, enabling farmers to track recurring issues and improvement measures.

The crop management feature provides stage-wise recommendations related to fertilizer application, crop care practices, and best farming methods. Farmers receive guidance tailored to different growth stages such as sowing, seedling, flowering, and harvesting. The irrigation guidance feature assists farmers in planning efficient water usage by suggesting appropriate watering schedules based on crop type, soil condition, and seasonal factors, thereby promoting water conservation and healthy crop growth.

The system includes a dedicated government schemes module that provides detailed information on agricultural subsidies, financial assistance programs, and welfare schemes. This feature ensures that farmers are aware of available benefits and understand eligibility criteria and application procedures. Additionally, the integration of RSS feeds allows the system to display real-time agricultural news, weather alerts, and policy updates, keeping farmers informed about current developments.

Other supporting features include farmer registration and login, a personalized dashboard, data storage using SQLite, and a simple web-based interface developed using Python Flask. The system is lightweight, cost-effective, and suitable for deployment in rural areas. These features work together to enhance agricultural productivity, increase farmer income, and empower tribal farmers through accessible digital technology.

SYSTEM DESIGN AND DEVELOPMENT



3. SYSTEM DESIGN AND DEVELOPMENT

System design and development form the backbone of the Smart Agribot for Tribal Farmers project. This phase translates the conceptual requirements of the system into a structured technical framework that defines how data will be processed, stored, and presented to users. The design ensures that the system is modular, scalable, and easy to maintain. The development process follows a systematic approach where each module such as user management, pest detection, crop guidance, irrigation scheduling, harvesting support, government schemes, and RSS integration is implemented independently while maintaining integration within a unified architecture.

The system is developed using Python Flask as the backend framework and SQLite as the database, ensuring lightweight deployment and suitability for rural environments with limited infrastructure. Proper validation mechanisms, multilingual support, and structured data handling techniques are incorporated during development to enhance usability and reliability. The design emphasizes simplicity, accuracy, and efficiency to meet the practical needs of tribal and small-scale farmers.

3.1 FILE DESIGN

File design refers to the structured organization of program files, templates, database files, and static resources that collectively form the Smart Agribot system. Proper file design ensures that the application remains organized, maintainable, and easy to extend in the future. The system follows a modular directory structure separating backend logic, templates, static assets, and database configuration.

Main files

The main application logic is stored in the primary Python file, typically named `app.py`, which contains route definitions, database connections, business logic, and RSS integration functions. This file acts as the central controller of the application and manages interactions between the user interface and the database.

Templates files

The templates folder contains all HTML files responsible for rendering the front-end interface. Files such as `base.html`, `login.html`, `register.html`, `dashboard.html`, `pest.html`, `irrigation.html`, `crop_management.html`, `harvest.html`, and `schemes.html` are organized systematically. The `base.html` file serves as the master template that includes common navigation menus and layout structures, ensuring design consistency across all pages. Other templates extend this base layout, improving maintainability and reducing redundancy.

Static folder files

The static folder stores CSS files, images, and optional JavaScript files used for styling and client-side interactivity. CSS files ensure consistent formatting, spacing, and responsive design across the system.

Database files

The database file, typically `agribot.db` (SQLite), stores all structured data including user records, pest guidance data, irrigation rules, crop stages, harvesting information, schemes, action history, and RSS source links. SQLite is chosen because it is lightweight, serverless, and suitable for small-scale deployment.

The file design also ensures separation of concerns. Backend logic remains independent of presentation logic, and database operations are clearly defined within structured functions. This modular design improves debugging, enhances scalability, and allows future integration of advanced features such as AI-based pest detection or image uploads.

The file design of the Smart Agribot system ensures clarity, modularity, efficient data handling, and maintainability, forming a strong foundation for reliable system development.

3.2 INPUT DESIGN

Input design plays a crucial role in ensuring the accuracy, reliability, and usability of the Smart Agribot system. It focuses on how data is collected from users and how it is validated before processing. Since the system is designed for tribal and small-scale farmers, the input forms are kept simple, structured, and easy to understand. Dropdown menus, validation checks, and required field constraints are implemented to reduce input errors. The system ensures that only relevant and standardized data is accepted, thereby improving the quality of recommendations generated by the backend modules. Proper input design enhances user confidence, minimizes confusion, and ensures efficient interaction between the user and the system.

User Registration Form

The user registration form collects essential information required to create a farmer account in the system. The form includes fields such as name, email, mobile number, location, preferred language, and password. Mandatory validation ensures that no required field is left empty. Email format validation ensures correct email structure, while mobile number validation restricts input to valid numeric values with appropriate length. The preferred language selection allows users to choose between English and Tamil, enabling multilingual functionality. Password encryption is implemented at the backend to ensure security. This form establishes the foundation for personalized system access and secure authentication.

User Login Form

The login form allows registered farmers to securely access the system. It collects username or email and password as inputs. Input validation ensures that empty credentials are not accepted. The backend verifies the credentials using hashed password comparison to prevent unauthorized access. Upon successful authentication, session management is initiated to maintain user login state. This form ensures secure entry into the application and protects user data.

Pest Detection Form

The pest detection form is designed to help farmers identify crop-related pest problems. The input fields include crop selection and symptom selection. Both inputs are presented through dropdown menus to prevent incorrect text entries. Only relevant symptoms are displayed based on the selected crop, reducing confusion. Once the user submits the form, the backend processes the selected inputs and retrieves appropriate pest identification and control measures. This structured input design ensures accurate problem diagnosis and effective recommendation delivery.

Crop Management Form

The crop management form allows farmers to obtain stage-wise guidance for specific crops. The form includes crop selection and growth stage selection. Growth stages such as sowing, vegetative stage, flowering stage, and harvesting stage are displayed as selectable options. The use of dropdown lists standardizes input values and ensures consistency in database queries. The form design ensures that farmers receive precise recommendations tailored to the crop's current growth phase.

Irrigation Management Form

The irrigation form collects inputs related to crop type, season, and soil type. These fields are presented as dropdown menus to ensure valid combinations. The form is designed in such a way that only relevant seasons and soil types are displayed based on the selected crop. This prevents invalid selections and eliminates unnecessary error messages. After submission, the system retrieves irrigation schedules specific to the selected parameters. The structured input approach improves water management recommendations and enhances decision-making accuracy.

Harvest Guidance Form

The harvest guidance form collects the selected crop type to provide information about maturity indicators, harvesting methods, and post-harvest handling techniques. Since harvesting guidance is crop-specific, a single crop dropdown is sufficient to retrieve detailed recommendations. The simplicity of this form ensures quick access to important harvesting information without overwhelming the user with excessive inputs.

Government Schemes Form

The government schemes module typically requires minimal input. Farmers can either browse available schemes or select a scheme category if filtering is provided. The system displays eligibility criteria, benefits, required documents, and application procedures based on the selected scheme. The input structure ensures easy navigation and improves awareness of financial support programs.

The input design of the Smart Agribot system emphasizes simplicity, accuracy, multilingual accessibility, and structured data validation. By carefully designing each form with proper validation and dropdown-based input mechanisms, the system minimizes errors and ensures that farmers receive precise and relevant agricultural guidance.

3.3 OUTPUT DESIGN

Output design refers to the manner in which processed information is presented to the users of the Smart Agribot system. The primary objective of output design is to ensure that the generated recommendations and information are displayed clearly, accurately, and in a user-friendly format. Since the system is intended for tribal and small-scale farmers, outputs are designed to be simple, easy to read, and language-friendly. The output screens avoid technical jargon and present agricultural guidance in a structured and understandable manner. Proper formatting, clear headings, and multilingual content presentation improve readability and usability.

Pest detection module

The pest detection module generates outputs based on the selected crop and symptoms. The output includes the identified pest name, prevention methods, and control

measures. The results are displayed in a structured format with separate sections for problem identification and solution guidance. If Tamil is selected as the preferred language, all pest details are displayed in Tamil; otherwise, English content is shown. This ensures clarity and eliminates confusion among farmers.

Crop management module

The crop management module produces stage-wise agricultural recommendations. Once a farmer selects a crop and growth stage, the system retrieves the relevant guidance and displays it clearly on the screen. The output includes instructions related to fertilizer application, crop care, and preventive practices. Information is organized into readable paragraphs rather than technical data tables, ensuring ease of understanding for farmers.

Irrigation management module

The irrigation management module generates irrigation schedules based on crop type, season, and soil conditions. The output design ensures that only valid and relevant irrigation schedules are displayed. The irrigation recommendation is shown prominently with a heading such as “Irrigation Schedule” to attract attention. The system avoids displaying unnecessary error messages, thereby maintaining a clean interface. The information is concise yet informative, enabling farmers to quickly understand watering frequency and management practices.

Harvest guidance module

The harvest guidance module provides outputs that include maturity indicators, harvesting techniques, and post-harvest handling steps. These outputs are organized into separate sections to improve clarity. Farmers can easily identify when the crop is ready for harvest and how to handle produce after harvesting to maintain quality and reduce losses. Multilingual output ensures better comprehension.

Government schemes module

The government schemes module displays scheme name, eligibility criteria, benefits, required documents, and application procedures in a structured format. The information is presented clearly without overwhelming the user. Headings and subheadings are used to separate different scheme components, ensuring better readability.

RSS news module

The RSS news module displays real-time agricultural news fetched dynamically from external sources. Each news item is presented with a title, publication date, and a clickable link directing users to the original website. This ensures farmers stay informed about current agricultural developments and weather alerts.

Dashboard output

The dashboard output consolidates recent activities, recommendations, and news in a personalized view. Farmers can review previous queries and results, enabling better decision-making and reference. The output design ensures consistency in layout, font styling, and spacing across all modules.

The output design of the Smart Agribot system focuses on clarity, simplicity, multilingual accessibility, and structured presentation. It ensures that complex agricultural data is transformed into meaningful and easily understandable guidance, empowering farmers to make informed decisions effectively.

3.4 DATABASE DESIGN

Database design is a critical component of the Smart Agribot for Tribal Farmers system, as it ensures structured storage, efficient retrieval, and secure management of agricultural and user-related data. The system uses SQLite as the backend database due to its lightweight nature, ease of integration with Python Flask, and suitability for rural or small-scale deployment environments. The database is designed following relational principles, where data is organized into multiple logically structured tables with appropriate primary keys and foreign key relationships to maintain data integrity.

Users table

The core table in the database is the Users table, which stores farmer registration details such as user ID, name, email, mobile number, location, preferred language, password hash, and account creation date. The user ID serves as the primary key and uniquely identifies each farmer. Passwords are stored in encrypted format to ensure security and prevent unauthorized access. The preferred language field enables multilingual functionality across the system.

Pest Guidance table

The Pest Guidance table stores structured information related to crop-specific pest problems. It contains fields such as crop name, symptom key, symptom description in English and Tamil, pest name in both languages, prevention methods, and control measures. This design allows the system to retrieve appropriate pest solutions based on selected crop and symptom combinations. The separation of English and Tamil fields ensures efficient multilingual content handling.

Crop Management table

The Crop Management table maintains stage-wise agricultural recommendations. It includes crop name, growth stage identifiers, stage names in English and Tamil, and recommendation content in both languages. This table supports precise retrieval of guidance based on crop type and current growth stage. The structure ensures organized storage and easy scalability if new crops or stages are added in the future.

Irrigation Rules table

The Irrigation Rules table is designed to manage irrigation schedules based on crop, season, and soil type combinations. It includes fields such as crop name, season key, season labels (English and Tamil), soil key, soil labels, and irrigation schedule details in both languages. The composite matching of crop, season, and soil type ensures accurate water management recommendations. This relational structure prevents invalid data combinations and enhances query efficiency.

Harvest Guidance table

The Harvest Guidance table stores information about maturity indicators, harvesting methods, and post-harvest handling practices for different crops. Separate fields for English and Tamil content support multilingual output. The table structure ensures that harvesting recommendations can be retrieved efficiently based on selected crop type.

Government Schemes table

The Government Schemes table contains detailed information about agricultural welfare programs, including scheme names, eligibility criteria, benefits, required documents, and application procedures in both languages. This centralized storage ensures that farmers can easily access scheme-related information without relying on external sources.

Action History table

The Action History table records user interactions such as pest queries, irrigation searches, and crop management requests. It stores user ID, action type, input parameters, output recommendations, selected language, and timestamp. This table helps maintain usage history and supports personalized dashboard functionality.

RSS Sources table

The RSS Sources table stores the URLs of agricultural news feeds. Unlike other tables, this table only stores feed links, while actual news content is dynamically fetched during dashboard loading. This design reduces database size and ensures real-time updates.

3.5 SYSTEM DEVELOPMENT

The Smart Agribot for Tribal Farmers is divided into multiple functional modules to ensure systematic operation and ease of use. Each module is designed to address specific agricultural needs and collectively provide an integrated support system for farmers.

- User Registration and Authentication Module
- Multilingual Support Module
- Pest Detection Module
- Crop Management Module
- Irrigation Management Module
- Harvesting Guidance Module
- Government Schemes Module
- RSS Feed Module
- Dashboard and History Module

3.5.1 Description of Modules

User Registration and Authentication Module

The User Registration and Authentication Module allows farmers to create an account and securely log in to the system. During registration, farmers can enter basic

details such as name, location, and preferred language. This module ensures secure access to the application and enables personalized services for each user.

Multilingual Support Module

The Multilingual Support Module enables farmers to interact with the system in their preferred regional language. All system messages, recommendations, and interface elements are displayed in the selected language, improving accessibility and understanding among tribal farmers with limited literacy or exposure to English-based applications.

Pest Detection Module

The Pest Detection Module assists farmers in identifying pest-related problems by allowing them to select crop types and observed symptoms. Based on the provided inputs, the system generates suitable pest identification and control recommendations. This module helps reduce crop damage by promoting early detection and appropriate corrective measures.

Crop Management Module

The Crop Management Module provides stage-wise guidance for different crops. Farmers receive recommendations related to fertilizer usage, crop care practices, and preventive measures during various growth stages such as sowing, growth, flowering, and harvesting. This ensures better crop health and improved yield.

Irrigation Management Module

The Irrigation Management Module helps farmers plan efficient water usage by providing crop-specific and season-based irrigation recommendations. This module supports optimal watering schedules, reduces water wastage, and promotes sustainable irrigation practices.

Harvesting Guidance Module

The Harvesting Guidance Module offers information on the right time and methods for harvesting crops. It provides indicators for maturity, harvesting techniques, and post-harvest handling tips, helping farmers maximize output quality and reduce losses.

Government Schemes Module

The Government Schemes Module provides information about various agricultural schemes, subsidies, and welfare programs. Farmers can access details related to eligibility, benefits, required documents, and application procedures, ensuring better awareness and utilization of government support.

RSS Feed Module

The RSS Feed Module displays real-time agricultural news, weather alerts, and policy updates by fetching information from reliable sources. This module keeps farmers informed about recent developments affecting agriculture.

Dashboard and History Module

The Dashboard and History Module presents a personalized overview of recent activities, recommendations, and alerts. It allows farmers to view previous pest reports, crop records, and saved information for future reference.

TESTING AND IMPLEMENTATION

4. TESTING AND IMPLEMENTATION

INTRODUCTION

Testing and implementation are essential phases in the development of the Smart Agribot for Tribal Farmers system. After system design and coding are completed, the application must be thoroughly tested to ensure that all modules function correctly and efficiently. Testing helps identify errors, logical flaws, performance issues, and security vulnerabilities before the system is deployed for real-world use. Since the Smart Agribot is designed to assist tribal farmers in making agricultural decisions, it is critical that the recommendations generated by the system are accurate, reliable, and easy to access. Implementation refers to the process of deploying the tested system into a working environment where users can interact with it. Proper deployment and maintenance ensure long-term stability, scalability, and continuous improvement of the application.

4.1 TESTING

Testing ensures that the Smart Agribot system meets functional and non-functional requirements. Various types of testing were performed to verify system reliability and performance.

Unit Testing

Unit testing involves testing individual components or modules independently to ensure they function correctly. In this project, each route and database query was tested separately.

S.No	Test Case ID	Module	Test Input	Expected Output	Actual Output	Status
1	UT01	User Registration	Valid name, email, mobile, password	Account created successfully	Account stored in database	Passed

2	UT02	Login Module	Valid username and password	User redirected to dashboard	Dashboard displayed	Passed
3	UT03	Pest Detection	Crop = Rice, Symptom = Leaf Spot	Correct pest name and control displayed	Correct solution shown	Passed
4	UT04	Irrigation Module	Crop, Season, Soil selected	Correct irrigation schedule displayed	Schedule displayed	Passed
5	UT05	Harvest Module	Crop selected	Harvest guidance displayed	Proper harvest details shown	Passed

Integration Testing

Integration testing verifies whether different modules interact correctly with each other. It ensures that data flows properly between the frontend, backend, and database.

S.No	Test Case ID	Modules Involved	Test Input	Expected Output	Actual Output	Status
1	IT01	Registration + Login	New user credentials	User able to login	Login successful	Passed
2	IT02	Irrigation + Database	Valid crop-season-soil	Correct data retrieved	Data matched database	Passed
3	IT03	Pest + History	Submit pest query	Query saved in history	History updated	Passed

4	IT04	Dashboard + RSS	Load dashboard	Latest news displayed	News fetched dynamically	Passed
---	------	-----------------	----------------	-----------------------	--------------------------	--------

System Testing

System testing evaluates the entire application as a whole to ensure all functionalities work together properly.

S.No	Test Case ID	Process	Expected Output	Actual Output	Status
1	ST01	Complete workflow (Login → Pest → Irrigation)	Smooth execution	All modules functioned properly	Passed
2	ST02	Access all modules	No broken links	All pages loaded correctly	Passed

Functional Testing

Functional testing checks whether each feature operates according to system requirements.

S.No	Test Case ID	Feature	Test Input	Expected Output	Actual Output	Status
1	FT01	Multilingual Support	Switch to Tamil	Content displayed in Tamil	Tamil content shown	Passed
2	FT02	Government Schemes	Select scheme	Scheme details displayed	Details shown properly	Passed

3	FT03	Dropdown Filtering	Select crop in irrigation	Only relevant seasons/soils shown	Relevant options displayed	Passed
---	------	--------------------	---------------------------	-----------------------------------	----------------------------	--------

Validation Testing

Validation testing ensures that input fields accept only valid data.

S.No	Test Case ID	Field	Test Input	Expected Output	Actual Output	Status
1	VT01	Email	Invalid email format	Error message displayed	Validation triggered	Passed
2	VT02	Mobile	Letters instead of numbers	Input rejected	Error shown	Passed
3	VT03	Required Fields	Leave fields blank	Submission blocked	Validation message displayed	Passed

Security Testing

Security testing ensures that unauthorized access is prevented and user data remains protected.

S.No	Test Case ID	Scenario	Test Input	Expected Output	Actual Output	Status
1	SCT01	Invalid Login	Wrong password	Access denied	Login blocked	Passed
2	SCT02	Unauthorized Access	Access dashboard without login	Redirect to login	Redirect successful	Passed

Performance Testing

Performance testing evaluates system speed and responsiveness under normal usage.

S.No	Test Case ID	Scenario	Expected Output	Actual Output	Status
1	PT01	Load dashboard with RSS	Page loads quickly	Loaded within few seconds	Passed
2	PT02	Multiple irrigation queries	System stable	No crash observed	Passed

Usability Testing

Usability testing ensures that the system is easy to understand and navigate for farmers.

S.No	Test Case ID	Scenario	Expected Output	Actual Output	Status
1	UTB01	Farmer selects pest solution	Easy navigation	User completed task easily	Passed
2	UTB02	Irrigation dropdown filtering	No irrelevant options	Only valid options shown	Passed

Compatibility Testing

Compatibility testing verifies that the system works across different browsers and devices.

S.No	Test Case ID	Environment	Expected Output	Actual Output	Status
1	CT01	Google Chrome	Proper display	Layout correct	Passed
2	CT02	Mobile browser	Responsive layout	Adjusted properly	Passed

The testing process confirmed that the Smart Agribot system functions reliably and meets user requirements.

4.2 IMPLEMENTATION

Implementation refers to the process of installing and configuring the Smart Agribot system for operational use. The system is implemented using Python Flask as the server-side framework and SQLite as the database. The development environment includes Windows operating system with required Python libraries such as Flask and feedparser installed. After completing coding and testing, the application is executed locally using the Flask development server. The database is initialized with required tables and seed data for pest guidance, crop management, irrigation rules, harvest guidance, schemes, and RSS sources.

User accounts are created through the registration module, and the system becomes ready for interaction. During implementation, attention is given to configuring session management, database connections, and multilingual functionality to ensure smooth performance. The lightweight nature of Flask and SQLite makes the system easy to implement without requiring complex server infrastructure.

4.3 DEPLOYMENT

Deployment involves making the Smart Agribot system accessible to users in a real-world environment. The application can be deployed on a local server, institutional server, or cloud platform depending on requirements. For small-scale rural deployment, the system can run on a basic computer or local network server. For broader access, it can be deployed on cloud platforms such as PythonAnywhere or similar hosting services.

During deployment, necessary configurations such as setting environment variables, securing secret keys, and ensuring internet connectivity for RSS fetching are completed. The database file is securely stored to prevent data loss. Proper backup mechanisms can be implemented to maintain data safety. Deployment ensures that farmers can access the system through a web browser without installing additional software.

4.4 MAINTENANCE

Maintenance ensures the long-term stability and effectiveness of the Smart Agribot system. Regular updates to agricultural data such as pest information, irrigation rules, and government schemes are required to keep the system relevant. RSS feed URLs must be monitored to ensure they remain active and functional. Periodic database backups should be performed to prevent data loss. System performance should be monitored to identify potential issues such as slow loading times or broken links. Security updates and bug fixes must be applied whenever necessary. As new agricultural practices or technologies emerge, additional modules or data entries can be incorporated into the system. Maintenance ensures continuous improvement and sustainability of the application.

CONCLUSION

5. CONCLUSION

The Smart Agribot for Tribal Farmers project successfully demonstrates how information and communication technology can be integrated into agriculture to support tribal and small-scale farmers. The system provides a structured and user-friendly digital platform that assists farmers throughout the crop lifecycle, including pest detection, crop management, irrigation scheduling, harvesting guidance, government scheme awareness, and agricultural news updates. By combining multilingual support with rule-based agricultural recommendations, the application bridges the gap between traditional farming practices and modern technological solutions.

The use of Python Flask and SQLite ensures that the system remains lightweight, cost-effective, and suitable for deployment in rural environments with limited infrastructure. The multilingual interface enhances accessibility and usability, particularly among tribal farmers who may face language barriers in existing agricultural portals. The integration of dynamic RSS feeds ensures that farmers remain updated with real-time agricultural news and weather alerts. Overall, the system contributes to informed decision-making, improved productivity, better resource utilization, and enhanced income stability. The Smart Agribot project highlights the potential of digital innovation in strengthening food security and empowering rural farming communities.

FUTURE ENHANCEMENTS

- The system can be enhanced by integrating image-based pest detection using machine learning techniques to automatically identify diseases from crop photos.
- Weather API integration can be added to provide real-time localized weather forecasts and rainfall predictions for better irrigation planning.
- An SMS notification system can be implemented to send alerts about irrigation schedules, pest outbreaks, and government scheme deadlines directly to farmers' mobile phones.
- An admin module can be developed to manage agricultural datasets, update scheme information, and monitor user activity.
- Voice-based interaction in regional languages can be introduced to assist farmers with limited literacy levels.
- Market price integration can be added to provide daily crop price updates from nearby markets to support better selling decisions.
- Offline mode functionality can be developed to allow access to basic guidance in areas with limited internet connectivity.
- Data analytics features can be incorporated to analyze farmer usage patterns and improve recommendation accuracy.
- Mobile application development can be considered to extend accessibility through Android-based platforms.

BIBLIOGRAPHY

BIBLIOGRAPHY

Reference Books

1. Laudon, K. C., & Laudon, J. P. (2020). Management Information Systems: Managing the Digital Firm. Pearson Education.
2. Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
3. Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.
4. Korth, H. F., Silberschatz, A., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.
5. Tanenbaum, A. S., & Van Steen, M. (2017). Distributed Systems: Principles and Paradigms. Pearson Education.
6. Reddy, T. Y., & Reddi, G. H. S. (2012). Principles of Agronomy. Kalyani Publishers.
7. Gupta, P. K. (2015). Crop Production and Management. Agro Bios (India).
8. Satapathy, S. C. (Ed.). (2021). Artificial Intelligence in Agriculture: A Practical Approach. Springer Nature.
9. Raj, S. (2020). Chatbots and Conversational AI: From Beginner to Expert. BPB Publications.

Reference Websites

1. Ministry of Agriculture and Farmers Welfare, Government of India. <https://agricoop.nic.in>
2. Indian Council of Agricultural Research (ICAR). <https://icar.org.in>
3. National Informatics Centre (NIC). <https://www.nic.in>
4. Food and Agriculture Organization of the United Nations (FAO). <https://www.fao.org>
5. Government of Tamil Nadu Agriculture Department. <https://www.tnagrisnet.tn.gov.in>
6. Flask Official Documentation. <https://flask.palletsprojects.com>
7. SQLite Official Documentation. <https://www.sqlite.org>

APPENDICES

APPENDICES

A. DATA FLOW DIAGRAM

A data flow diagram is graphical tool used to describe and analyze movement of data through a system. These are the central tool and the basis from which the other components are developed. The transformation of data from input to output, through processed, may be described logically and independently of physical components associated with the system. These are known as the logical data flow diagrams. The physical data flow diagrams show the actual implements and movement of data between people, departments and workstations. A full description of a system actually consists of a set of data flow diagrams.

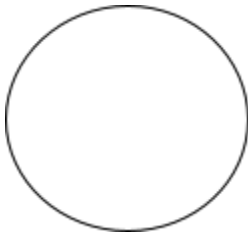
Using two familiar notations Yourdon, Gane and Sarson notation develops the data flow diagrams. Each component in a DFD is labeled with a descriptive name. Process is further identified with a number that will be used for identification purpose. in the context level diagram is exploded into other process at the first level DFD.

The idea behind the explosion of a process into more process is that understanding at one level of detail is exploded into greater detail at the next level. This is done until further explosion is necessary and an adequate amount of detail is described for analyst to understand the process. Larry Constantine first developed the DFD as a way of expressing system requirements in a graphical from, this lead to the modular design. A DFD is also known as a “bubble Chart” has the purpose of clarifying system requirements and identifying major transformations that will become programs in system design. So it is the starting point of the design to the lowest level of detail.

DFD SYMBOLS

- In the DFD, there are four symbols
- A square defines a source(originator) or destination of system data
- An arrow identifies data flow. It is the pipeline through which the information flows

- A circle or a bubble represents a process that transforms incoming data flow into outgoing data flows.



Process which transforms data flow.



Source or Destination of data



Data flow



Data Store

Constructing a DFD

Several rules of thumb are used in drawing DFD'S:

- Process should be named and numbered for an easy reference. Each name should be representative of the process.
- The direction of flow is from top to bottom and from left to right. Data traditionally flow from source to the destination although they may flow back to the source. One way to indicate this is to draw long flow line back to a source. An alternative way is to repeat the source symbol as a destination. Since it is used more than once in the DFD it is marked with a short diagonal.
- When a process is exploded into lower level details, they are numbered.

- The names of data stores and destinations are written in capital letters. Process and dataflow names have the first letter of each word capitalized

A DFD typically shows the minimum contents of data store. Each data store should contain all the data elements that flow in and out.

Salient features of DFD's

- The DFD shows flow of data, not of control loops and decision are controlled considerations do not appear on a DFD.
- The DFD does not indicate the time factor involved in any process whether the dataflow take place daily, weekly, monthly or yearly.

TYPES OF DATA FLOW DIAGRAMS

- Current Physical
- Current Logical
- New Logical
- New Physical

Current physical

In Current Physical DFD process label include the name of people or their positions or the names of computer systems that might provide some of the overall system-processing label includes an identification of the technology used to process the data. Similarly data flows and data stores are often labels with the names of the actual physical media on which data are stored such as file folders, computer files, business forms or computer tapes.

Current logical

The physical aspects at the system are removed as much as possible so that the current system is reduced to its essence to the data and the processors that transform them regardless of actual physical form.

New physical

The new physical represents only the physical implementation of the new system.

New logical

This is exactly like a current logical model if the user were completely happy with the user were completely happy with the functionality of the current system but had problems with how it was implemented typically through the new logical model will differ from current logical model while having additional functions, absolute function removal and inefficient flows recognized.

RULES GOVERNING THE DFD'S

Process

- No process can have only outputs.
- No process can have only inputs. If an object has only inputs than it must be a sink.
- A process has a verb phrase label.

Data store

- Data cannot move directly from one data store to another data store, a process must move data.
- Data cannot move directly from an outside source to a data store, a process, which receives, must move data from the source and place the data into data store
- A data store has a noun phrase label.

Source or sink

Source is the origin or destination of data.

- Data cannot move direly from a source to sink it must be moved by a process
- A source and /or sink has a noun phrase land

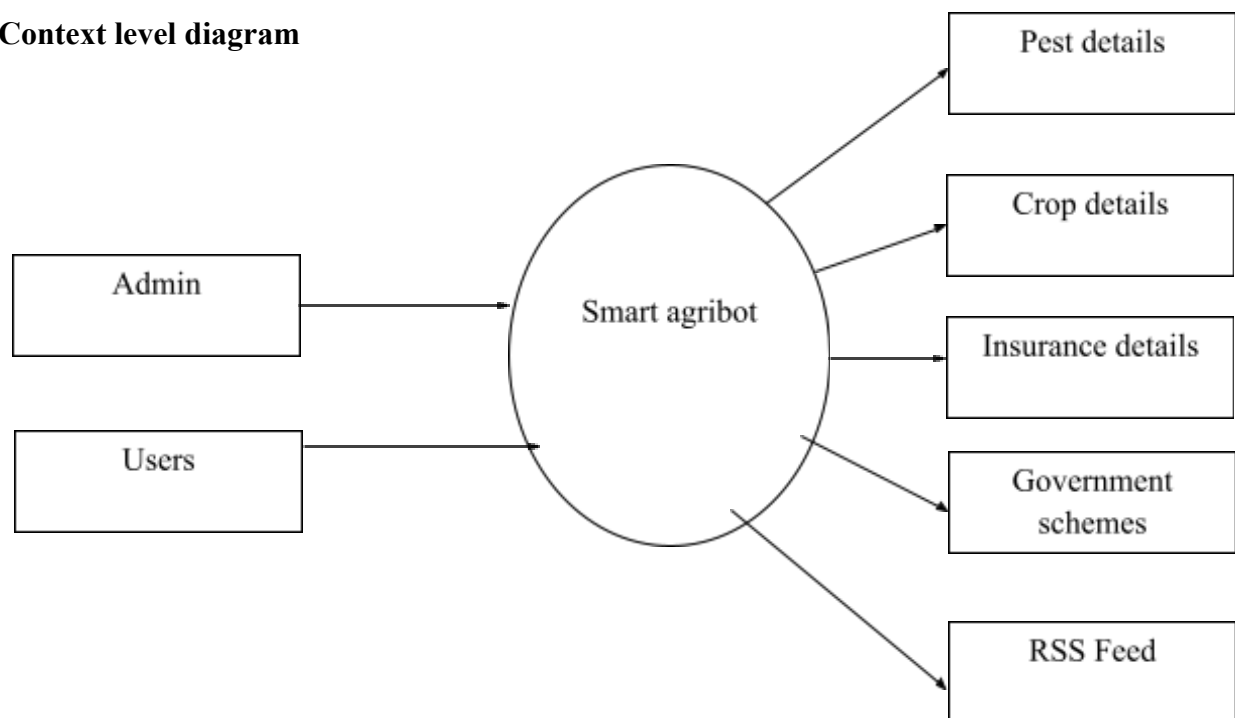
Data flow

- A Data Flow has only one direction of flow between symbols. It may flow in both directions between a process and a data store to show a read before an update. The

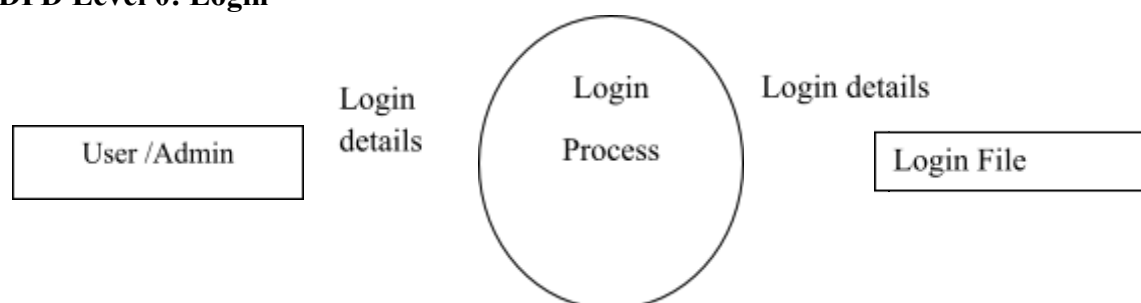
later is usually indicated however by two separate arrows since these happen at different type.

- A join in DFD means that exactly the same data comes from any of two or more different processes data store or sink to a common location.
- A data flow cannot go directly back to the same process it leads. There must be at least one other process that handles the data flow produce some other data flow returns the original data into the beginning process.

Context level diagram

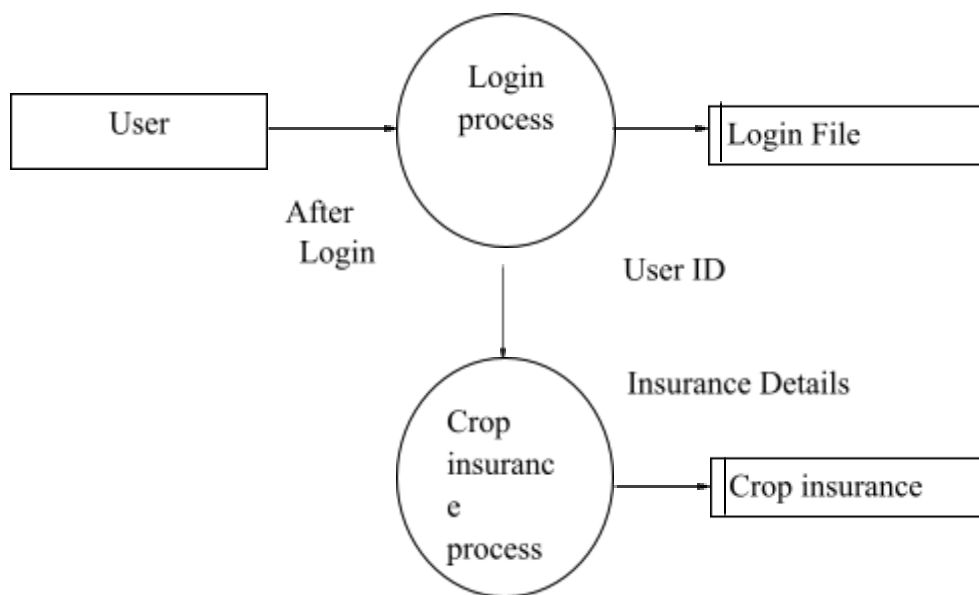


DFD Level 0: Login

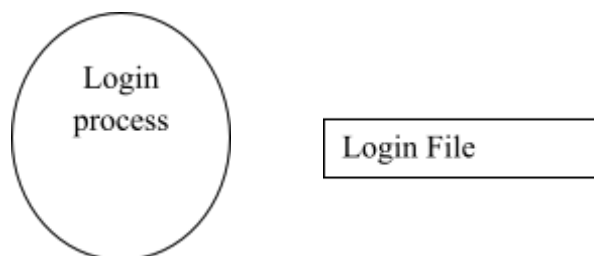


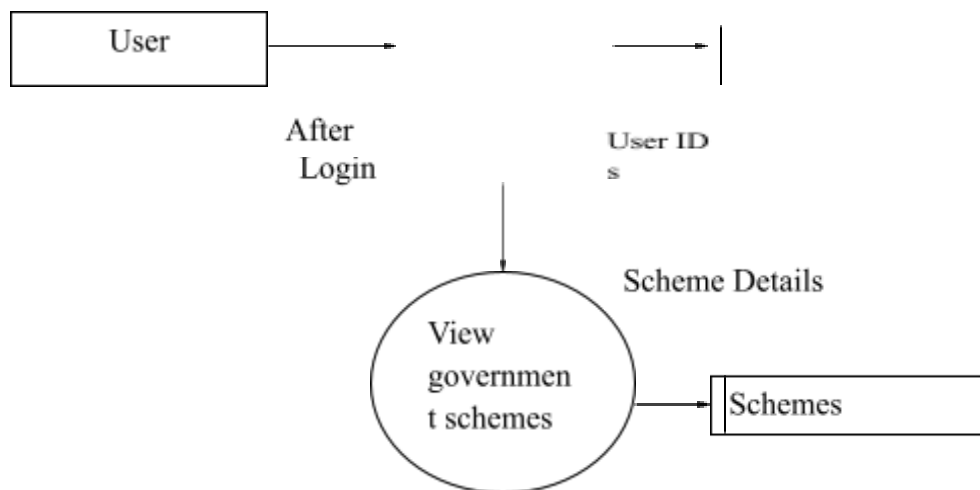


DFD Level 1: crop insurance details

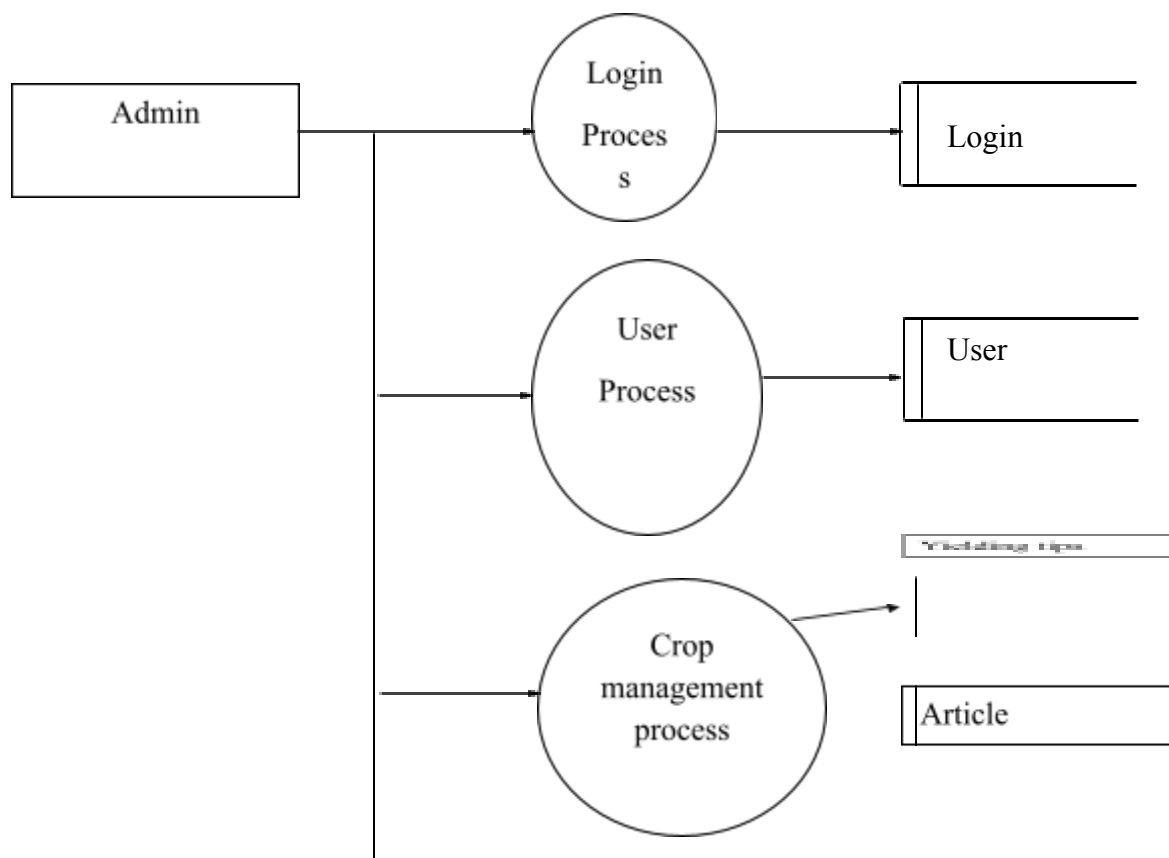


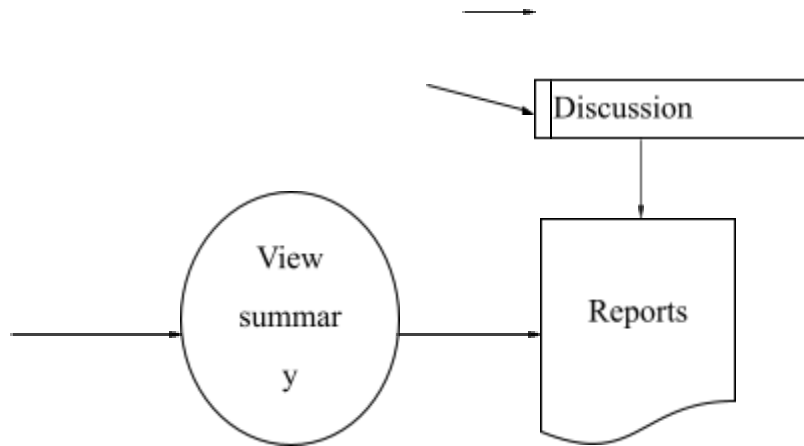
DFD Level 1: schemes process





Level 2: Admin





C. SAMPLE CODING

App.py

```
import os
```

```
import re
```

```
import sqlite3
```

```
from datetime import datetime
```

```
from functools import wraps
```

```
import json
```

```
import csv
```

```
from io import StringIO, BytesIO
```

```
from flask import Response, send_file
```

```
import feedparser
```

```
from flask import (
```

```
    Flask, render_template, request, redirect, url_for,
```

```
    flash, session, jsonify
```

```
)
```

```
from werkzeug.security import generate_password_hash, check_password_hash
```

```
APP_DIR = os.path.abspath(os.path.dirname(__file__))
```

```
DB_PATH = os.path.join(APP_DIR, "agribot.db")
```

```
app = Flask(__name__)
```

```
app.secret_key = "change-this-to-a-strong-secret-key"
```

```
@app.context_processor
```

```
def inject_helpers():
```

```
    def crop_label(crop):
```

```
        lang = session.get("lang", "en")
```

```
        if lang == "ta":
```

```
            return CROP_TA.get(crop, crop)
```

```
        return crop
```

```
    return dict(crop_label=crop_label)
```

```
# -----
```

```
# Database helpers
```

```
# -----
```

```
def get_db():
```

```
    conn = sqlite3.connect(DB_PATH)
```

```
    conn.row_factory = sqlite3.Row
```

```
    return conn
```



```

def init_db():

    conn = get_db()

    cur = conn.cursor()

    # Users

    cur.execute("""

CREATE TABLE IF NOT EXISTS users (

    id INTEGER PRIMARY KEY AUTOINCREMENT,

    name TEXT NOT NULL,

    email TEXT NOT NULL UNIQUE,

    phone TEXT NOT NULL UNIQUE,

    location TEXT,

    preferred_language TEXT NOT NULL DEFAULT 'en',

    password_hash TEXT NOT NULL,

    created_at TEXT NOT NULL

)

""")

    # Chat sessions

```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS chat_sessions (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    user_id INTEGER NOT NULL,
```

```
    title TEXT NOT NULL,
```

```
    created_at TEXT NOT NULL,
```

```
    FOREIGN KEY(user_id) REFERENCES users(id)
```

```
)
```

```
""")
```

```
# Chat messages
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS chat_messages (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    session_id INTEGER NOT NULL,
```

```
    sender TEXT NOT NULL CHECK(sender IN ('user','bot')),
```

```
    message TEXT NOT NULL,
```

```
    created_at TEXT NOT NULL,
```

```
    FOREIGN KEY(session_id) REFERENCES chat_sessions(id)
```

)

""")

Bilingual pest guidance

cur.execute("""

CREATE TABLE IF NOT EXISTS pest_guidance (

id INTEGER PRIMARY KEY AUTOINCREMENT,

crop TEXT NOT NULL,

symptom_key TEXT NOT NULL,

symptom_en TEXT NOT NULL,

symptom_ta TEXT NOT NULL,

pest_name_en TEXT NOT NULL,

pest_name_ta TEXT NOT NULL,

prevention_en TEXT NOT NULL,

prevention_ta TEXT NOT NULL,

control_en TEXT NOT NULL,

control_ta TEXT NOT NULL,

notes_en TEXT,

notes_ta TEXT

)

""")

Bilingual crop management

cur.execute("""

CREATE TABLE IF NOT EXISTS crop_management (

id INTEGER PRIMARY KEY AUTOINCREMENT,

crop TEXT NOT NULL,

stage_key TEXT NOT NULL,

stage_en TEXT NOT NULL,

stage_ta TEXT NOT NULL,

recommendation_en TEXT NOT NULL,

recommendation_ta TEXT NOT NULL

)

""")

Bilingual irrigation rules

cur.execute("""

CREATE TABLE IF NOT EXISTS irrigation_rules (

```
id INTEGER PRIMARY KEY AUTOINCREMENT,

crop TEXT NOT NULL,

season_key TEXT NOT NULL,

season_en TEXT NOT NULL,

season_ta TEXT NOT NULL,

soil_key TEXT NOT NULL,

soil_en TEXT NOT NULL,

soil_ta TEXT NOT NULL,

schedule_en TEXT NOT NULL,

schedule_ta TEXT NOT NULL

)

""")
```

```
# Bilingual harvesting
```

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS harvest_guidance (
```

```
id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
crop TEXT NOT NULL,
```

```
maturity_en TEXT NOT NULL,
```

```
maturity_ta TEXT NOT NULL,  
  
method_en TEXT NOT NULL,  
  
method_ta TEXT NOT NULL,  
  
post_en TEXT NOT NULL,  
  
post_ta TEXT NOT NULL  
  
)  
  
""")
```

Bilingual schemes

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS schemes (
```

```
    id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
    scheme_name_en TEXT NOT NULL,
```

```
    scheme_name_ta TEXT NOT NULL,
```

```
    eligibility_en TEXT NOT NULL,
```

```
    eligibility_ta TEXT NOT NULL,
```

```
    benefits_en TEXT NOT NULL,
```

```
    benefits_ta TEXT NOT NULL,
```

```
    documents_en TEXT NOT NULL,
```

```
documents_ta TEXT NOT NULL,  
  
apply_en TEXT NOT NULL,  
  
apply_ta TEXT NOT NULL  
  
)  
  
""")
```

Action history (bilingual)

```
cur.execute("""
```

```
CREATE TABLE IF NOT EXISTS action_history (  
  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
  
    user_id INTEGER NOT NULL,  
  
    action_type TEXT NOT NULL,      -- pest / crop / irrigation / harvest / scheme / news  
  
    input_json TEXT NOT NULL,      -- stored as JSON string  
  
    output_text TEXT NOT NULL,  
  
    lang TEXT NOT NULL DEFAULT 'en',  
  
    created_at TEXT NOT NULL,  
  
    FOREIGN KEY(user_id) REFERENCES users(id)  
  
)  
  
""")
```

RSS sources (name can be bilingual through UI labels; keep as is)

cur.execute("""

CREATE TABLE IF NOT EXISTS rss_sources (

id INTEGER PRIMARY KEY AUTOINCREMENT,

name TEXT NOT NULL,

url TEXT NOT NULL

)

""")

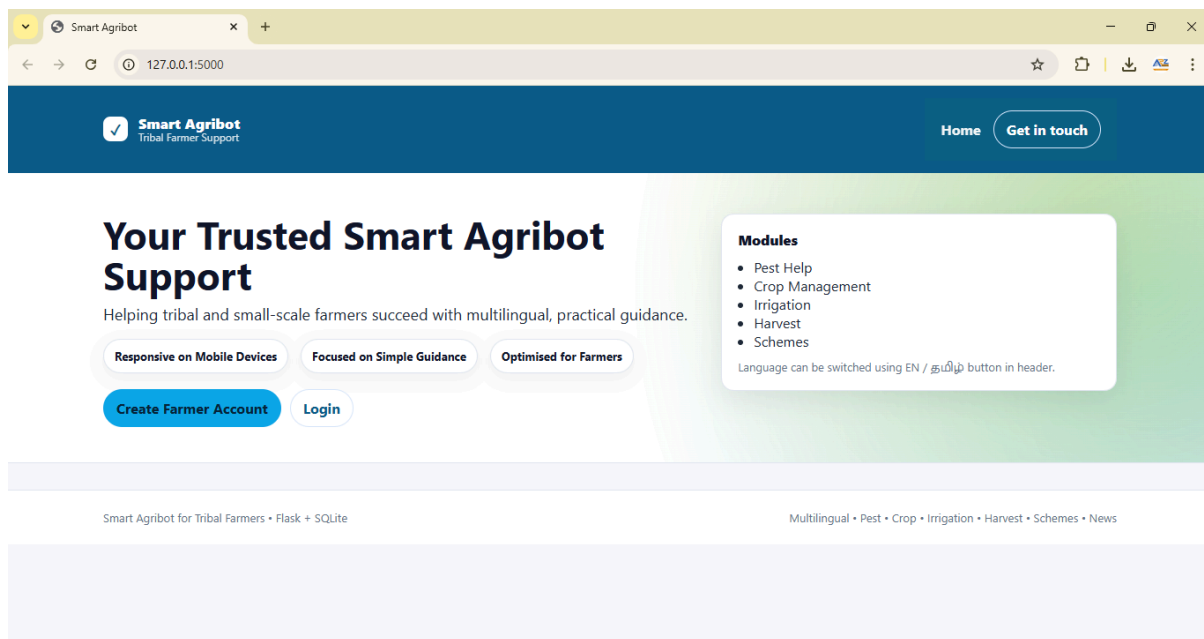
conn.commit()

seed_if_empty(conn)

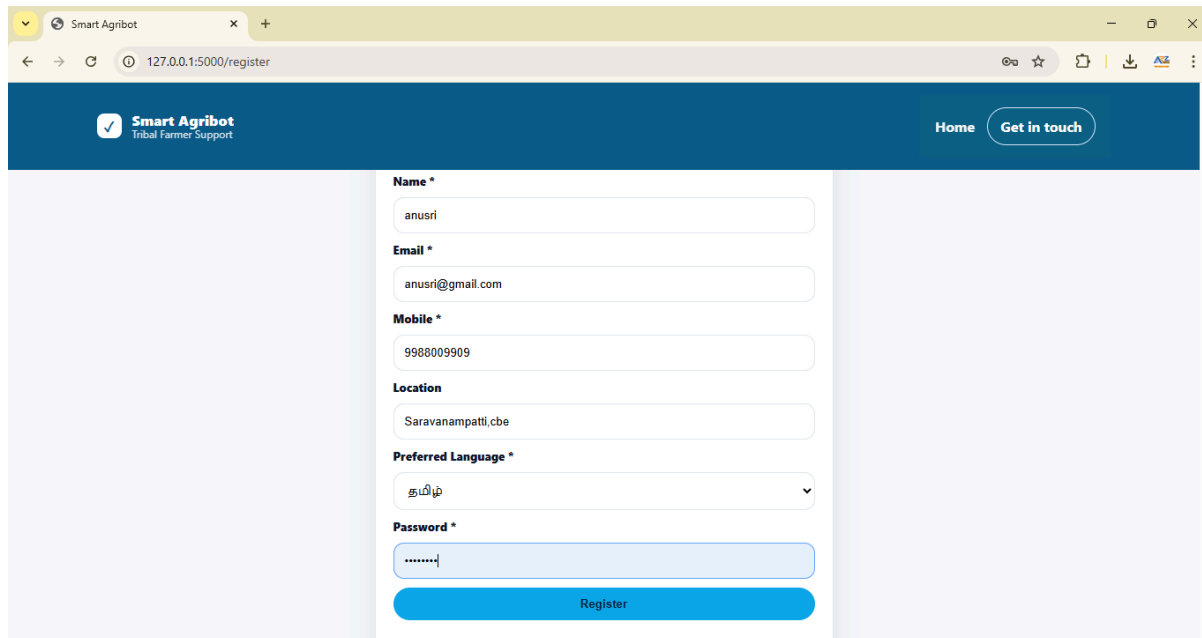
conn.close()

D. SAMPLE INPUT

Home page



Account creation



Smart Agribot Tribal Farmer Support

Home Get in touch

Name *

anusr

Email *

anusr@gmail.com

Mobile *

9988009909

Location

Saravanampatti.cbe

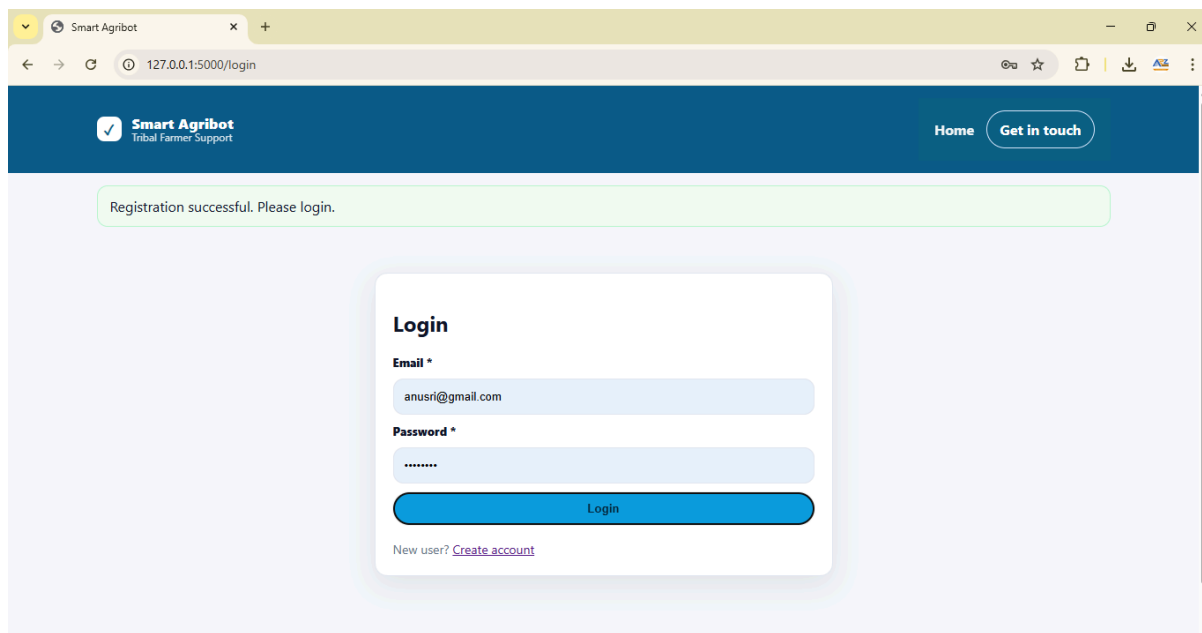
Preferred Language *

தமிழ்

Password *

Register

Login page



Smart Agribot Tribal Farmer Support

Home Get in touch

Registration successful. Please login.

Login

Email *

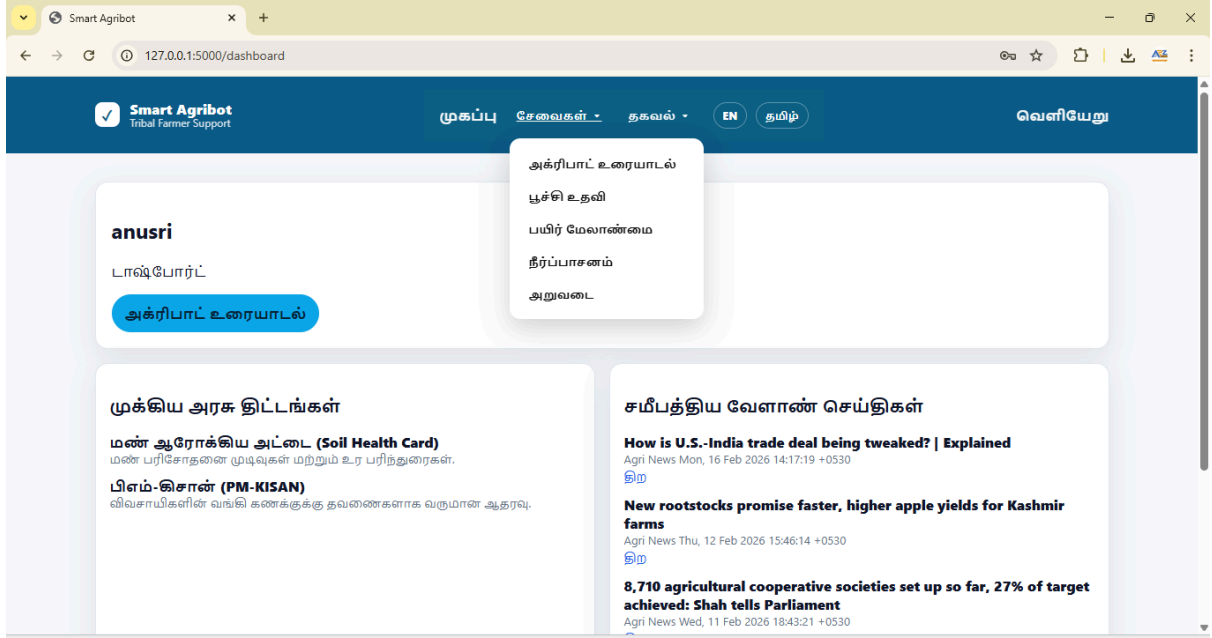
anusr@gmail.com

Password *

Login

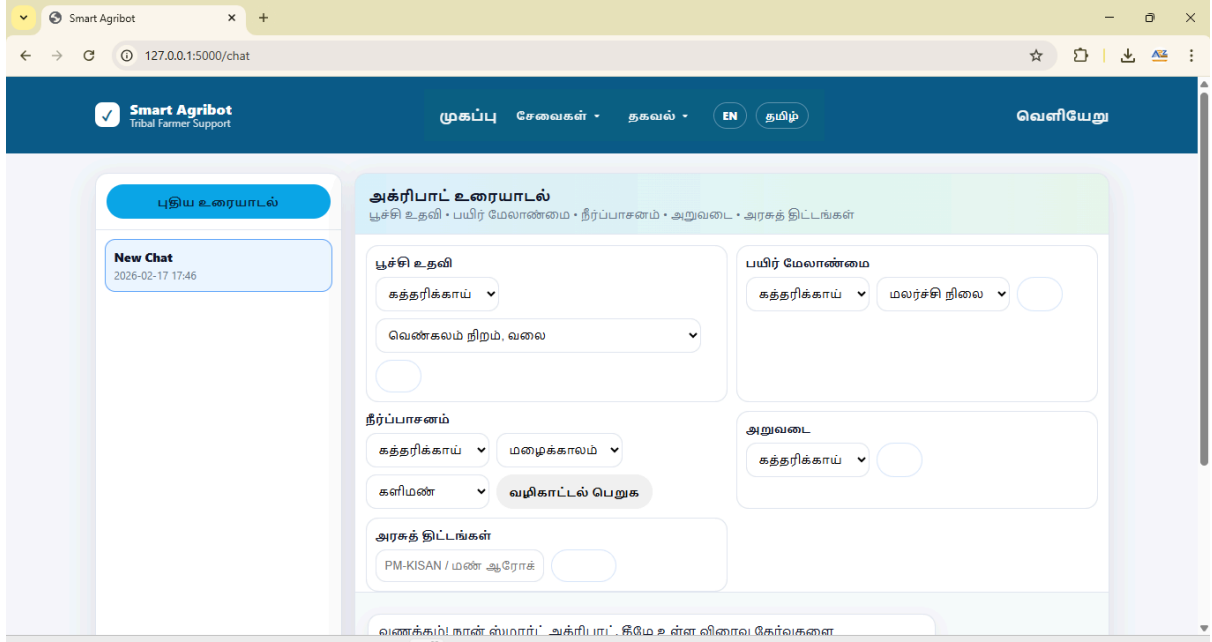
New user? [Create account](#)

User home with user preference

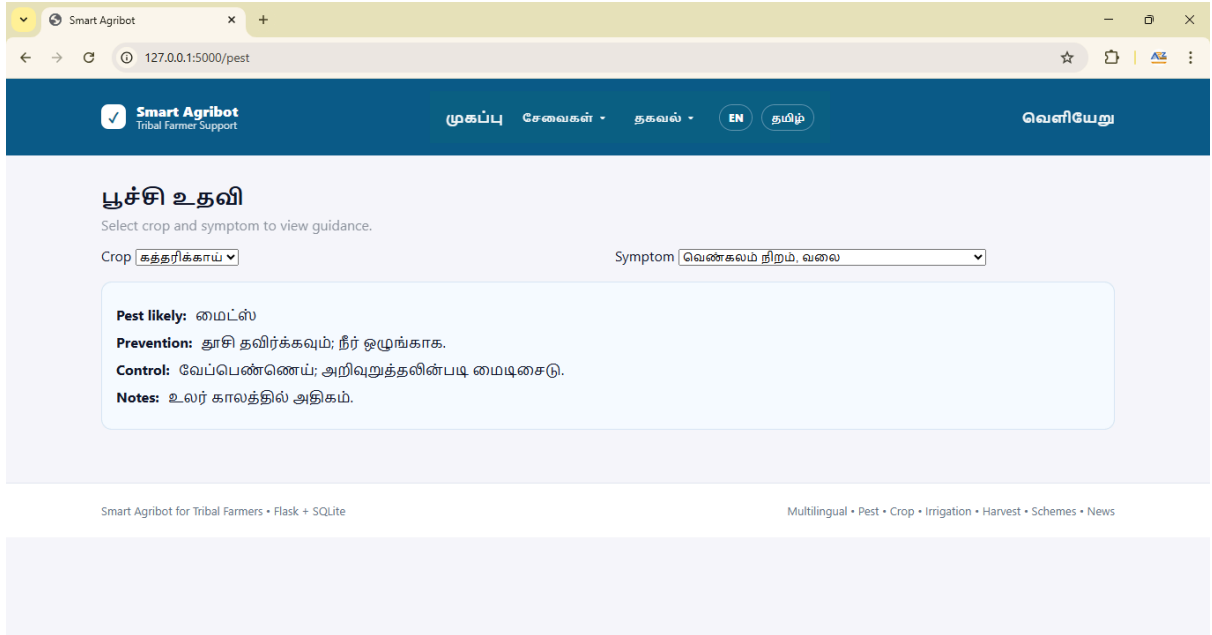


E. SAMPLE OUTPUT

Chat



Pest



Crop management

Smart Agribot
Tribal Farmer Support

Home Services Info EN தமிழ்

Logout

Crop Management

Crop Brinjal stage Sowing

Send

Prepare nursery; use treated seeds; raised beds.

Smart Agribot for Tribal Farmers • Flask • SQLite

Multilingual • Pest • Crop • Irrigation • Harvest • Schemes • News

Irrigation tips

Smart Agribot
Tribal Farmer Support

Home Services Info EN தமிழ்

Logout

Irrigation

Select crop first

Crop Brinjal Season Summer Soil Clay

Get guidance

Schedule: Irrigate once in 6 days.

Smart Agribot for Tribal Farmers • Flask • SQLite

Multilingual • Pest • Crop • Irrigation • Harvest • Schemes • News

Harvest tips

Smart Agribot
Tribal Farmer Support

HomeServicesInfoENதமிழ்Logout

Harvest

Crop: Brinjal

Send

Maturity signs: Fruits glossy and tender; seeds soft; avoid over-maturity.

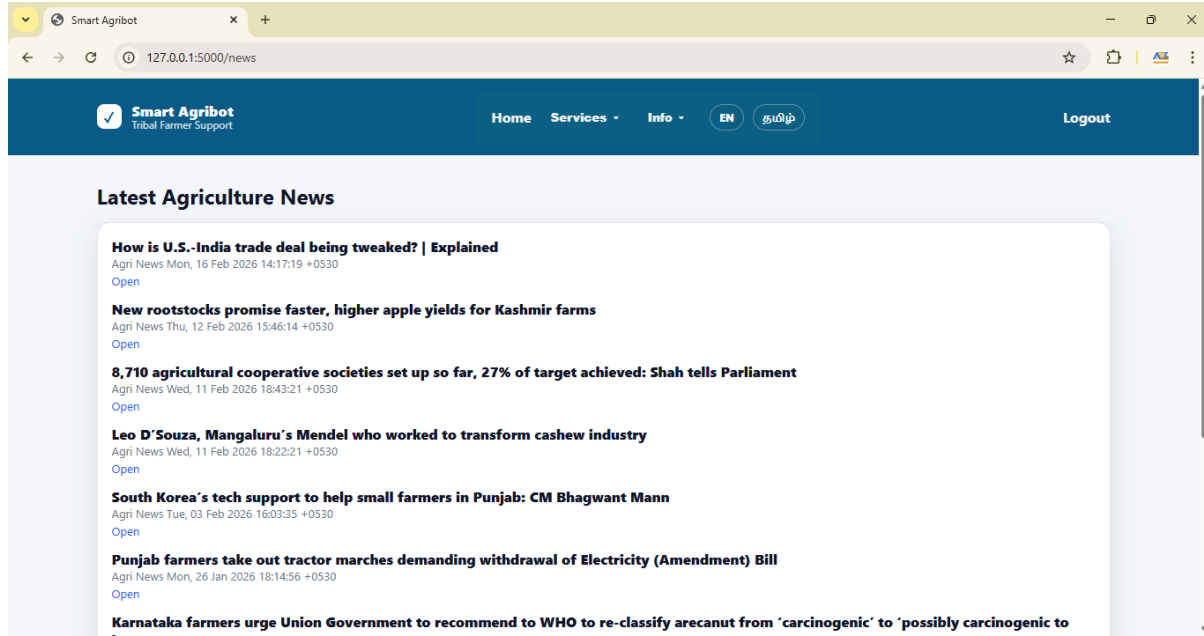
Harvest method: Harvest with knife/secateur; avoid bruising; pick frequently.

Post-harvest: Sort by size; keep ventilated crates; quick transport to market.

Smart Agribot for Tribal Farmers • Flask • SQLite

Multilingual • Pest • Crop • Irrigation • Harvest • Schemes • News

Latest news feed



History analysis

Smart Agribot
Tribal Farmer Support
Home
Services
Info
EN
தமிழ்
Logout

History

Filters + Export CSV / Export PDF

All
Pest
Crop
Irrigation
Harvest
Scheme
News
Export CSV
Export PDF

NEWS
Input: {count: 9}
Output
News opened

2026-02-17 17:55

HARVEST
Input: {crop: 'Brinjal'}
Output
Maturity signs: Fruits glossy and tender; seeds soft; avoid over-maturity.
Harvest method: Harvest with knife/secateur; avoid bruising; pick frequently.
Post-harvest: Sort by size; keep ventilated crates; quick transport to market.

2026-02-17 17:55

CROP

2026-02-17 17:55

Reports

